

第十二章 软件测试

本章要点

- 一、软件测试概述
- 二、白盒测试技术
- 三、黑盒测试技术
- 四、灰盒测试
- 五、软件测试步骤
- 六、软件测试周期
- 七、安全测试

1、软件测试概述

1.1 软件测试基本概念

- 测试是为了发现程序中的错误而执行程序的过程。无论怎样强调软件测试的重要性和它对软件可靠性的影响都不过分。

软件测试在软件生存周期中横跨两个阶段。

- 1、通常在编出每个模块之后就对它做必要的测试（称为**单元测试**），模块的编写者和测试者是同一个人，**编码和单元测试属于软件生存周期的同一个阶段**。
- 2、在这个阶段结束之后，对软件系统还要进行各种**综合测试**，这是软件生存周期中的另一个独立的阶段，通常由专门的测试人员承担这项工作。

基本概念

- 测试阶段的根本目的：尽可能多地发现并排除软件中潜藏的错误，最终把一个高质量的软件系统交给用户使用。
- 软件工程的测试阶段，测试人员努力设计出一系列测试方案，目的却是为了“破坏”已经建造好的软件系统--竭力证明程序中有错误不能按照预定要求正确执行。
- 测试一个大型程序所需要的创造力，事实上可能要超过设计那个程序所要求的创造力。

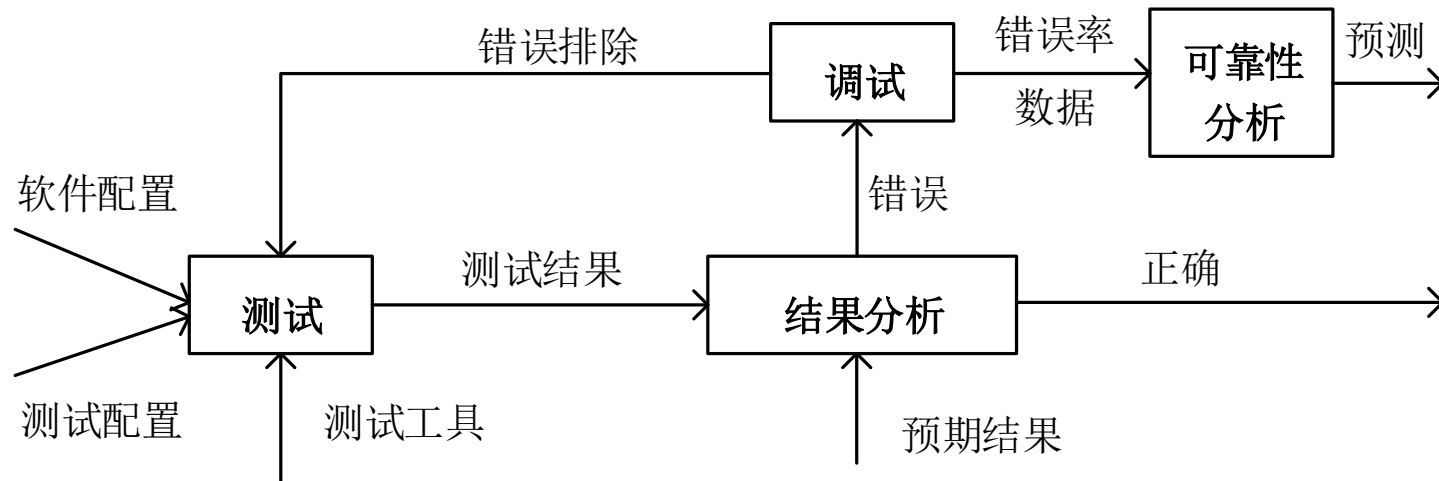
软件测试的目的

- 1、测试是为了发现程序中的错误而执行程序的过程；
- 2、好的测试方案是极可能发现迄今为止尚未发现的错误的测试方案；
- 3、成功的测试是发现了至今为止尚未发现的错误的测试。

● 测试的目的决定了测试方案的设计

- 1、**为了证明程序是正确的**而进行测试，就会设计一些不易暴露错误的的测试方案。
- 2、**为了发现程序中的错误**，就会力求设计出最能暴露错误的测试方案。从心理学的角度看，由程序的编写者自己进行测试是不恰当的。

软件测试对象和信息流



- 1) 软件配置：包括需求说明书，设计说明书，源程序清单等；
- 2) 测试配置：包括测试计划和测试方案。
- 3) 测试工具：winrunner，robot；



1.2测试文档

- 测试计划
- 测试规范
- 测试用例
- 测试报告
- 缺陷报告

1.3 软件测试的原则

- 应制定测试计划并严格执行，排除随意性；
- 应尽早地开始软件测试
- 避免程序员测试自己的程序
- 注重设计测试用例，测试数据不仅要选择合理的输入数据，还要选择不合理的输入数据；
- 对发现错误较多的程序段，应进行更深入的测试。在IBM370的某个操作系统中，47%的错误和缺陷集中在4%的代码中。
- 测试不能证明软件无错
- 软件缺陷的“免疫力”
- 全面记录每一个测试结果
- 长期保留测试用例等文档；
- 并非所有软件缺陷都修复

1.4 软件测试的类型

1. 静态测试和动态测试

- 静态测试是指不运行被测程序本身，仅通过分析或检查源程序的语法、结构、过程、接口等来检查程序的正确性。
- 静态测试可以对需求规格说明书、软件设计说明书、源程序做结构分析、流程图分析、符号执行来找错。
- 静态测试的方法，主要有手工检查与软件自动分析两大类。

2. 白盒测试和黑盒测试

- 白盒指的是清楚盒子内部的东西是如何运作的。
- 黑盒测试又称功能测试、数据驱动测试或基于规格说明的测试

2 白盒测试技术

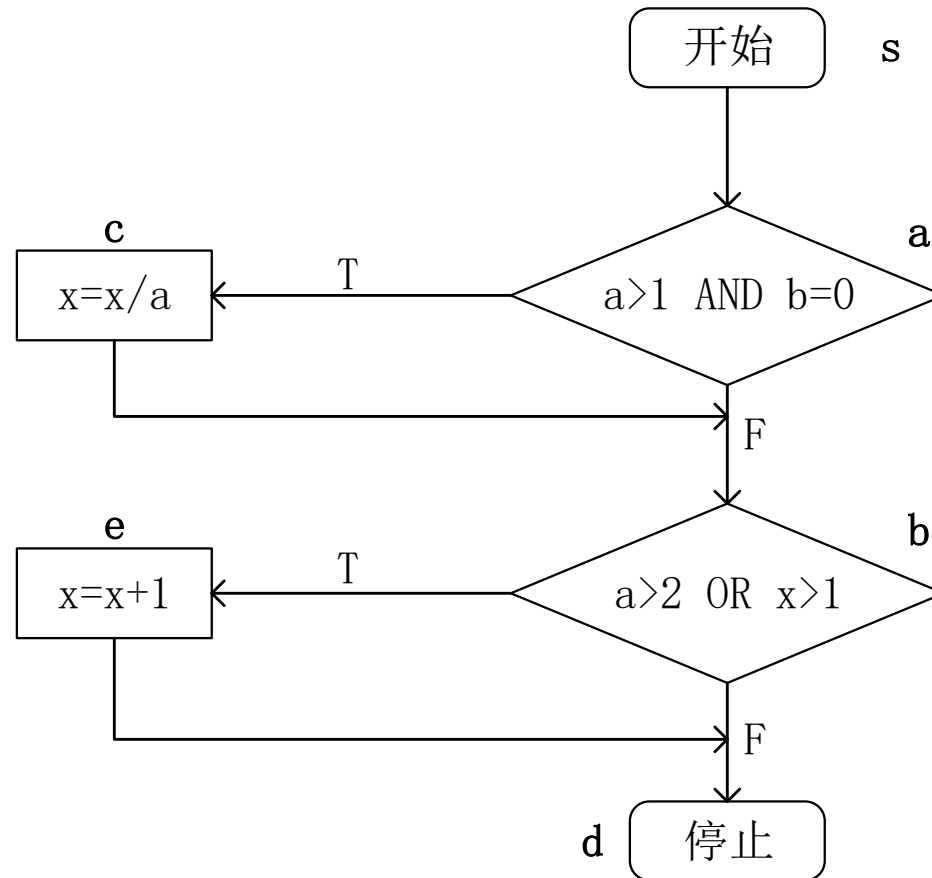
逻辑覆盖是设计白盒测试方案的一种技术。是对一系列测试过程的总称。

从覆盖源程序语句的详尽程度分析：

- 1、语句覆盖
- 2、判定覆盖
- 3、条件覆盖
- 4、判定/条件覆盖
- 5、条件组合覆盖
- 6、路径覆盖

软件测试方法

测试举例 (被测模块的流程图)



软件测试方法

1、语句覆盖

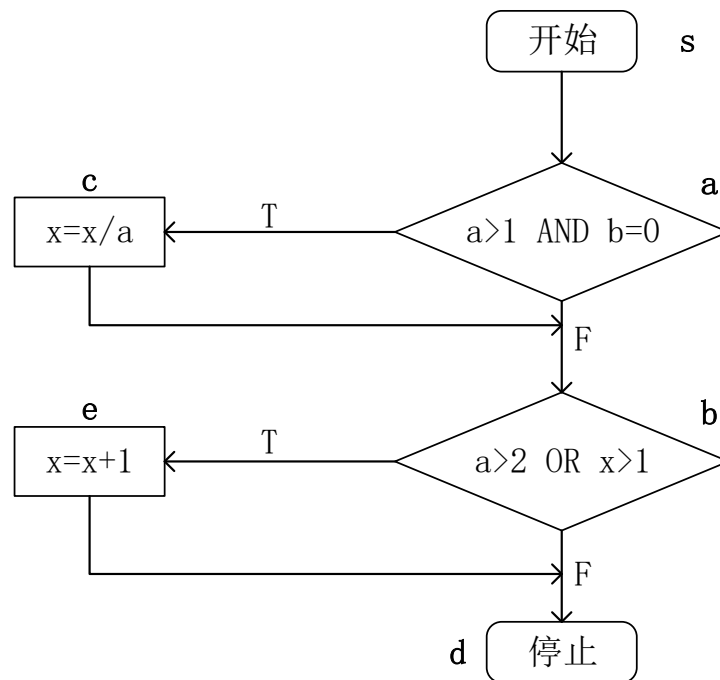
语句覆盖就是设计若干个测试用例，运行被测程序，使得每一可执行语句至少执行一次。

测试数据:

A=2, B=0, X=4

预期结果:

X=3.0



分析：1、只测试了条件为真的情况，当条件为假时处理有错误，语句测试不能发现。

2、只关心判定式的值，未测试每个条件取不同值情况。

3、如把AND错写成OR，把 $x>1$ 错写成 $x<1$ ，上述测试数据不能发现。

软件测试方法

2、判定覆盖

测试数据：

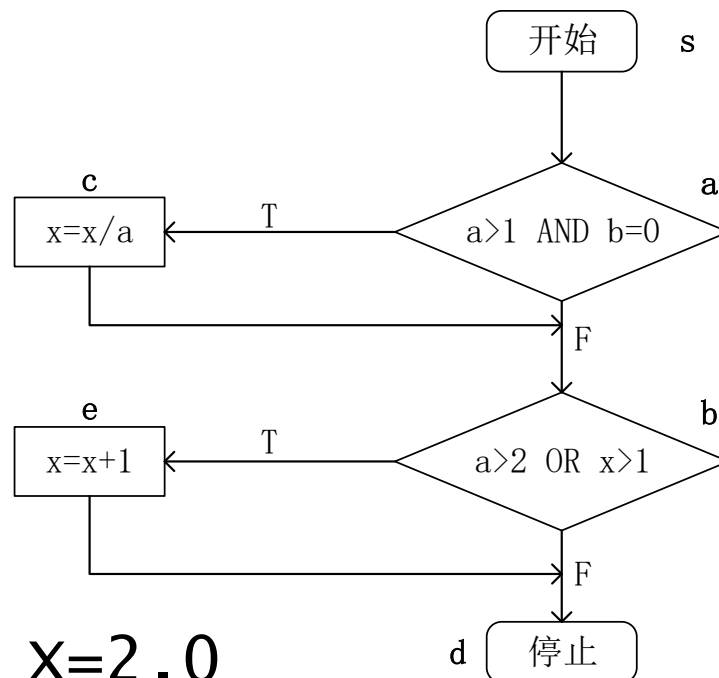
I: A=3, B=0, X=3
(覆盖sacbd)

II: A=2, B=1, X=1
(覆盖sabcd)

预期结果：

I: X=1.0

II : X=2.0



■判定覆盖就是设计若干个测试用例，运行被测程序，使得程序中每个判断的**取真分支和取假分支**至少经历一次。

分析：

- 1、判定覆盖比语句覆盖强，但逻辑覆盖程度还不高。
- 2、上述测试数据只覆盖了程序全部路径的一半。

软件测试方法

3、条件覆盖

• 条件覆盖就是设计若干个测试用例，运行被测程序，使得程序中每个判断的每个条件的可能取值至少执行一次。

a点: $A > 1, A \leq 1, B = 0, B \neq 0$;

b点: $A = 2, A \neq 2, X > 1, X \leq 1$;

测试数据:

I: $A = 2, B = 0, X = 4$; (sacbed)

($A > 1, B = 0, A = 2, X > 1$;))

II: $A = 1, B = 1, X = 1$; (sabd)

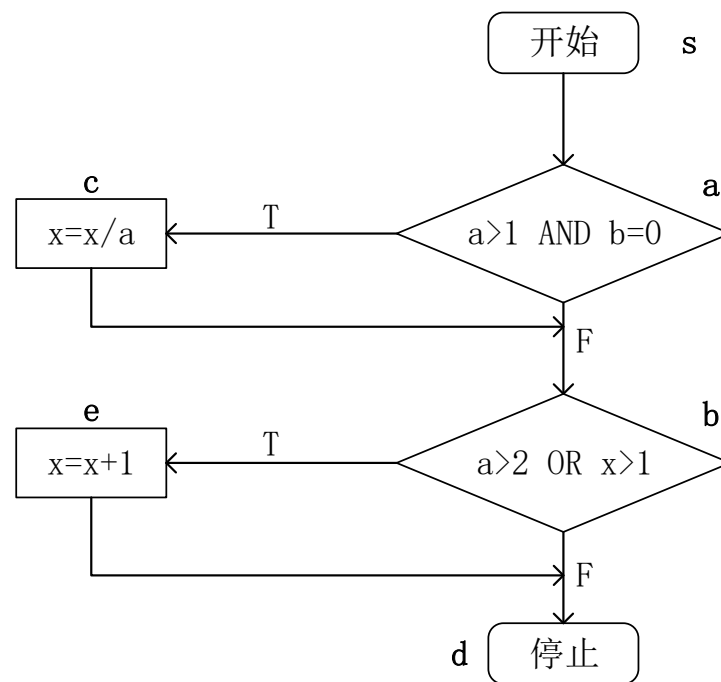
($A \leq 1, B \neq 0, A \neq 2, X \leq 1$;))

预期结果:

I: $X = 3.0$

II: $X = 1.0$

分析: 条件覆盖比判定覆盖强。



软件测试方法

4、判定/条件覆盖

测试数据：

I: A=2, B=0, X=4; (sacbed)

(A>1,B=0,A=2,X>1;

II: A=1, B=1, X=1; (sabd)

(A<=1,B!=0,A!=2,X<=1;

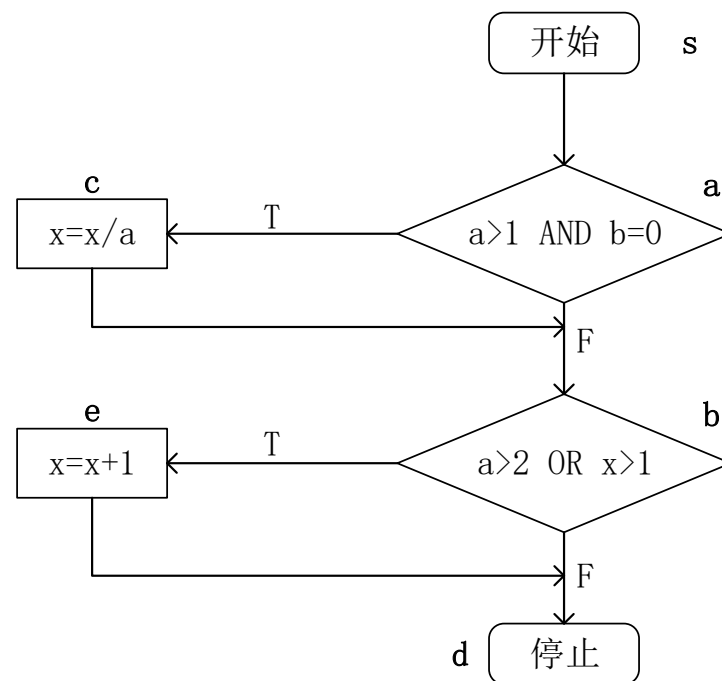
预期结果：

I: X=3.0

II : X=1.0

•判定/条件覆盖就是设计足够的测试用例，使得判断中每个条件的所有可能取值至少执行一次，每个判断的可能取值至少执行一次。

分析：判定/条件覆盖并不比条件覆盖强。



软件测试方法

5、条件组合覆盖

8种可能的条件组合如下：

- (1) $A > 1, B = 0$; (5) $A = 2, X > 1$;
- (2) $A > 1, B \neq 0$; (6) $A = 2, X \leq 1$;
- (3) $A \leq 1, B = 0$; (7) $A \neq 2, X > 1$;
- (4) $A \leq 1, B \neq 0$; (8) $A \neq 2, X \leq 1$;

测试数据：

I: $A=2, B=0, X=4$ (sacbed)

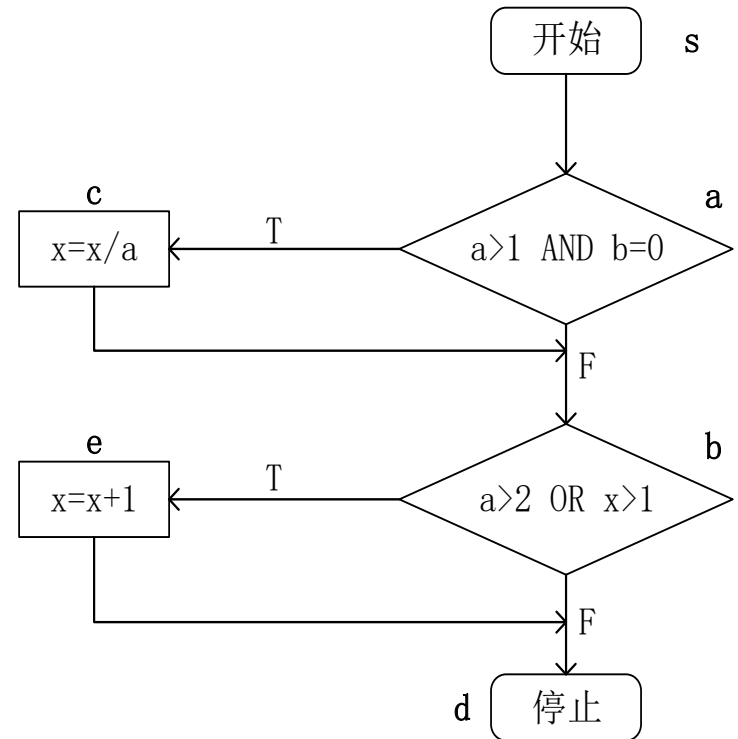
II: $A=2, B=1, X=1$ (sabed)

III: $A=1, B=0, X=2$ (sabed)

IV: $A=1, B=1, X=1$ (sabd)

条件组合覆盖就是设计足够的测试用例，运行被测程序，使得每个判断的所有可能的条件取值组合至少执行一次。

预期结果： I: $X=3.0$ II: $X=2.0$ III: $X=3.0$ IV: $X=1.0$



软件测试方法

6、路径覆盖

4条可能的路径:

1-2-3; 1-2-6-7; 1-4-5-3; 1-4-5-6-7。

测试数据:

I: A=1, B=1, X=1; (1-2-3)

II: A=1, B=1, X=2; (1-2-6-7)

III: A=3, B=0, X=1; (1-4-5-3)

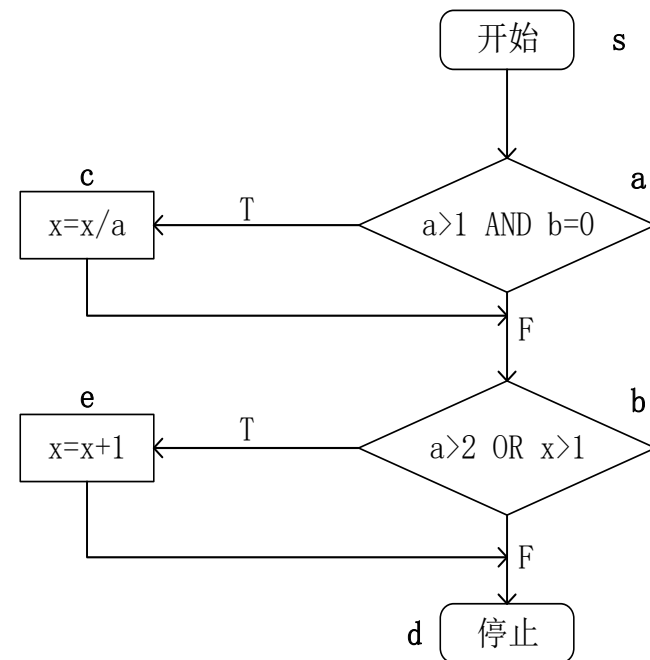
IV: A=2, B=0, X=4; (1-4-5-6-7)

预期结果:

I: X=3.0 II: X=2.0 III: X=3.0 IV: X=1.0

- 路径测试就是设计足够的测试用例，覆盖程序中所有可能的路径。

分析：路径覆盖是相当强的逻辑覆盖标准。和条件组合覆盖结合起来，可以设计出检错能力更强的测试数据。



3. 黑盒测试

- 这种方法是把测试对象看做一个黑盒子，测试人员完全不考虑程序内部的逻辑结构和内部特性，只依据程序的需求规格说明书，检查程序的功能是否符合它的功能说明。
- 黑盒测试又叫做功能测试或数据驱动测试。

3.1 等价类划分

- 等价类划分是一种典型的黑盒测试方法，使用这一方法时，完全不考虑程序的内部结构，只依据程序的规格说明来设计测试用例。
- 等价类划分方法把所有可能的输入数据，即程序的输入域划分成若干部分，然后从每一部分中选取少数有代表性的数据做为测试用例。
- 等价类：每类中的一个典型值在测试中的作用与这一类中的所有其他值的作用相同。

黑盒测试技术

- 等价类的划分有两种不同的情况：
 - ① 有效等价类：是指对于程序的规格说明来说，是合理的，有意义的输入数据构成的集合。
 - ② 无效等价类：是指对于程序的规格说明来说，是不合理的，无意义的输入数据构成的集合。
- 在设计测试用例时，要同时考虑有效等价类和无效等价类的设计。

黑盒测试技术

(2)如果规定了输入数据必须遵守的规则，则可以确立一个有效等价类（符合规则）和若干个无效等价类（从不同角度违反规则）。

例如，在Pascal语言中对变量标识符规定为“以字母打头的.....串”。那么所有以字母打头的构成有效等价类，而不在在此集合内（不以字母打头）的归于无效等价类。

黑盒测试技术

(3) 如果规定了输入数据的一组值，而且程序要对每个输入值分别进行处理。这时可为每一个输入值确立一个有效等价类，此外针对这组值确立一个无效等价类，它是所有不允许的输入值的集合。

例如，在教师上岗方案中规定对教授、副教授、讲师和助教分别计算分数，做相应的处理。因此可以确定4个有效等价类为教授、副教授、讲师和助教，一个无效等价类，它是所有不符合以上身分的人员的输入值的集合。

黑盒测试技术

在确立了等价类之后，建立等价类表，列出所有划分出的等价类，再从划分出的等价类中按以下原则选择测试用例：

(1) 设计一个新的测试用例，使其尽可能多地覆盖尚未被覆盖的有效等价类，重复这一步，直到所有的有效等价类都被覆盖为止；

(2) 设计一个新的测试用例，使其仅覆盖一个尚未被覆盖的无效等价类，重复这一步，直到所有的无效等价类都被覆盖为止。

3.2 边界值分析

人们从长期的测试工作经验得知，大量的错误是发生在输入或输出范围的边界上，而不是在输入范围的内部。因此**针对各种边界情况设计测试用例，可以查出更多的错误。**

通常，输入等价类和输出等价类的边界，就是应该着重测试的程序边界情况。

选取测试数据应该：

“刚好等于； 刚好小于； 刚好大于。”

某应用程序完成如下功能：

输入某年某月某日，判断这一天是这一年的第几天。

边界值分析

例如： 某应用程序完成如下
功能： 输入某年某月某
日， 判断这一天是这一
年的第几天。

程序C源代码如下：

```
main()
{
int day,month,year,sum,leap;
printf("\nplease input
      year,month,day\n");
scanf("%d,%d,%d",&year,
      &month,&day);
```

```
    switch(month)
    {
        case 1:sum=0; break;
        case 2:sum=31; break;
        case 3:sum=59; break;
        case 4:sum=90; break;
        case 5:sum=120; break;
        case 6:sum=151; break;
        case 7:sum=181; break;
        case 8:sum=212; break;
        case 9:sum=243; break;
        case 10:sum=273; break;
        case 11:sum=304; break;
        case 12:sum=334; break;
        default:printf("dataerror"); break;
    }
    sum=sum+day;
    printf("It is the %dth day.",sum);}
```

边界值分析

序号	输入条件	预期输出	实际输出
1	1900,1,1	It is the 1th day	It is the 1th day
2	1899,0,0	dataerror	dataerror
3	1901,2,2	It is the 33th day	It is the 33th day
4	2050,12,31	It is the 365th day	It is the 365th day
5	2049,11,30	It is the 334th day	It is the 334th day
6	2051,13,32	dataerror	dataerror
7			

软件测试

该程序是否有缺陷？这些缺陷可以用什么测试案例检测出来？

有缺陷，没有考虑闰年问题。可以用以下测试案例测试：

year=2000,month=3,day=1

预期输出： 6 1 实际输出： 6 0

修改缺陷部分，给出代码。

```
if(year%400==0||(year%4==0&&year%100!=0))
```

```
    /*判断是不是闰年*/
```

```
    leap=1;
```

```
    else
```

```
        leap=0;
```

```
    if(leap==1&&month>2)
```

```
        /*如果是闰年且月份大于2,总天数应该加一天*/
```

```
            sum++;
```

软件测试

测试数据及预期结果

序号	输入数据	预期输出	实际输出	说明
1	1900,1,1	It is the first day		刚好等于最小值
2	2050,12,31	It is the 365th day		刚好等于最大值
3	1899,1,1	输入年份超范围		年份刚小于最小值
4	2051,1,1	输入年份超范围		年份刚大于最大值
5	2000,0,1	输入月份超范围		月份刚小于最小值
6	2000,13,32	输入月份超范围		月份刚大于最大值
7	2000,1,0	输入日超范围		日刚小于最小值
8	2000,1,32	输入日超范围		日刚大于最大值
9	2000,3,1	It is the 61th day		闰年问题

3.3 错误推测

错误推测

基本思想：是列举出程序中可能有的错误和容易发生错误的特殊情况，并且根据他们选择测试方案。

经验告诉我们：在一段程序中已经发现的错误数往往和未发现的错误数成正比。

错误推测法很大程度上靠**直觉**和**经验**进行。

4. 灰盒测试

灰盒测试的定义

- * 1999年提出。单纯从名称上来看，灰盒测试是介于黑盒测试与白盒测试之间的一种测试方式。
- * 灰盒测试不像白盒那样详细、完整，但又比黑盒测试更关注程序的内部逻辑，常常是通过一些表征性的现象、事件、标志来判断内部的运行状态。
- * 灰盒测试多用于集成测试阶段，不仅关注输出、输入的正确性，同时也关注程序内部的情况。

灰盒测试

灰盒测试的定义

- * 灰盒测试与黑盒测试的区别

如果某软件包含多个模块，当使用黑盒测试时，只要关心整个软件系统的边界，无需关心软件系统内部各个模块之间如何协作。而如果使用灰盒测试，就需要关心模块与模块之间的交互。这是灰盒测试与黑盒测试的区别。

灰盒测试

灰盒测试的定义

- * 灰盒测试与白盒测试的区别

在灰盒测试中，无需关心模块内部的实现细节。对于软件系统的内部模块，灰盒测试依然把它当成一个黑盒来看待。而白盒测试还需要再深入地了解内部模块的实现细节。

灰盒测试

* 灰盒测试与单元测试的区别

首先，在进行单元测试时，需要写一些测试代码（即“桩代码”stub）。通常测试代码和被测试代码通常是同种语言（比如Java的单元测试通常也用Java来写），且测试代码和被测试代码的耦合很紧密。因此，单元测试通常由开发人员来完成的，测试人员的能力未必能胜任。

其次，单元测试的颗粒度会更细（会细到类一级、函数一级），而灰盒测试仅仅到模块一级。

灰盒测试

灰盒测试的优缺点

优点：

- 1、能够进行基于需求的覆盖测试和基于程序路径覆盖的测试；
- 2、测试结果可以对应到程序内部路径，便于bug的定位、分析和解决；
- 3、能够保证设计的黑盒测试用例的完整性，防止遗漏软件的一些不常用的功能或功能组合；
- 4、能够降低需求或设计不详细或不完整对测试造成的影响。

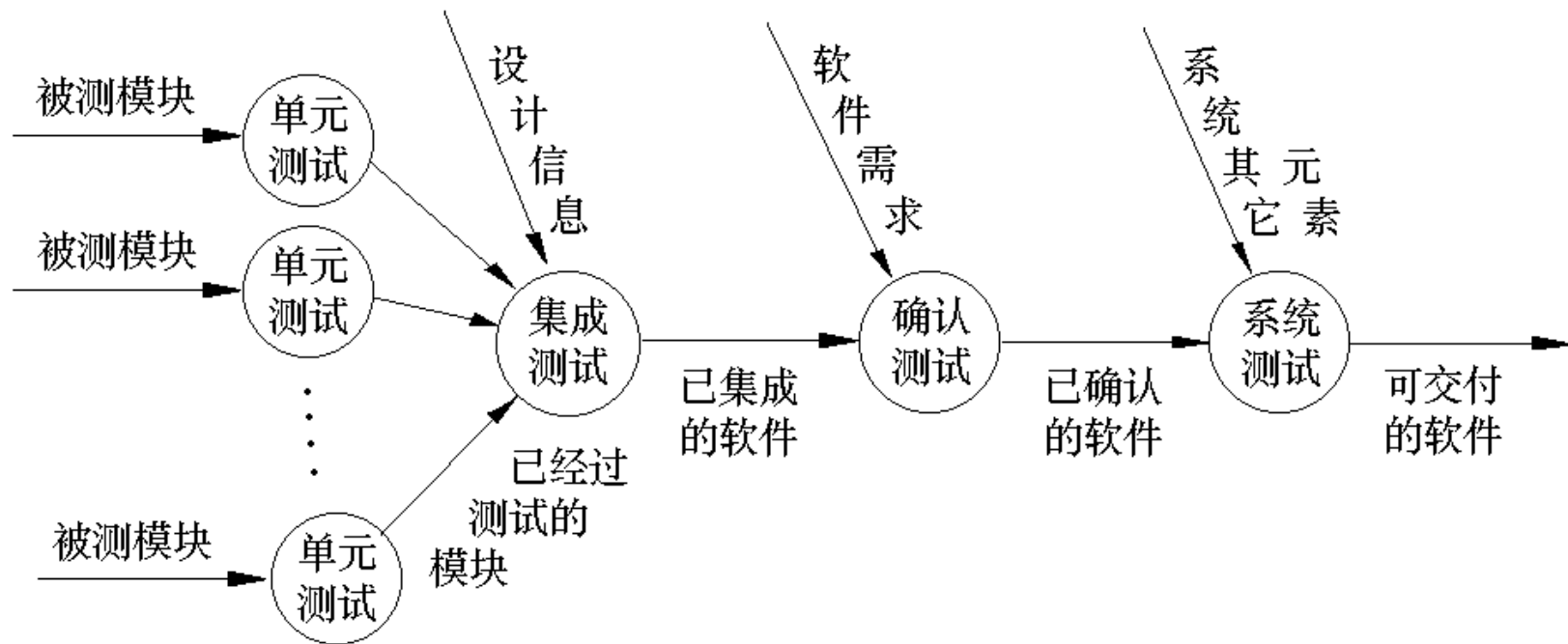
灰盒测试

灰盒测试的优缺点

缺点：

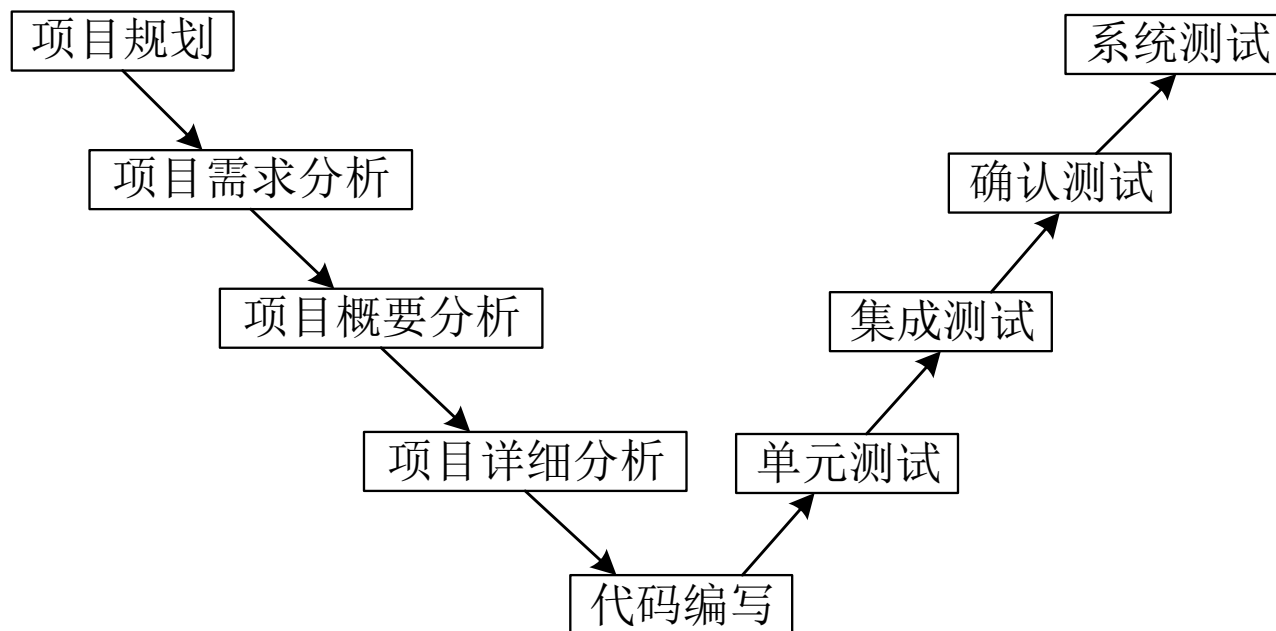
- 1、投入的时间比黑盒测试大概多20-40%的时间；
- 2、对测试人员的要求比黑盒测试高；灰盒测试要求测试人员清楚系统内部由哪些模块构成，模块之间如何协作。
- 3、不如白盒测试深入；
- 4、不适用于简单的系统。所谓的简单系统，就是简单到总共只有一个模块。由于灰盒测试关注于系统内部模块之间的交互。如果某个系统简单到只有一个模块，那就没必要进行灰盒测试了。

5. 软件测试步骤



5.1 软件测试V模型

- 软件测试V模型是在20 世纪80 年代后期提出的，目的是改进软件开发的效率和效果。
- V 模型从左到右，描述了基本的开发过程和测试行为，明确地标明了测试工程中存在的不同级别的测试以及测试阶段和开发过程各阶段的对应关系。



5.1 软件测试V模型

- V模型详细的描述了每个测试阶段所对应验证的对象，单元测试验收的对象是详细测试说明书、集成测试验证的对象是概要设计说明书，确认测试和系统测试验证的对象是需求说明书。

5.2 单元测试

单元测试又称模块测试。每个程序模块完成一个相对独立的子功能，所以可以对该模块进行单独的测试。

在单元测试期间主要评价模块的下述5个特性：

- 模块的接口
- 局部数据结构
- 重要的执行通路
- 出错处理通路
- 影响上述各方面特性的边界条件

单元测试

通常单元测试在编码阶段进行，并经过人工检查和计算机测试两种类型的测试。

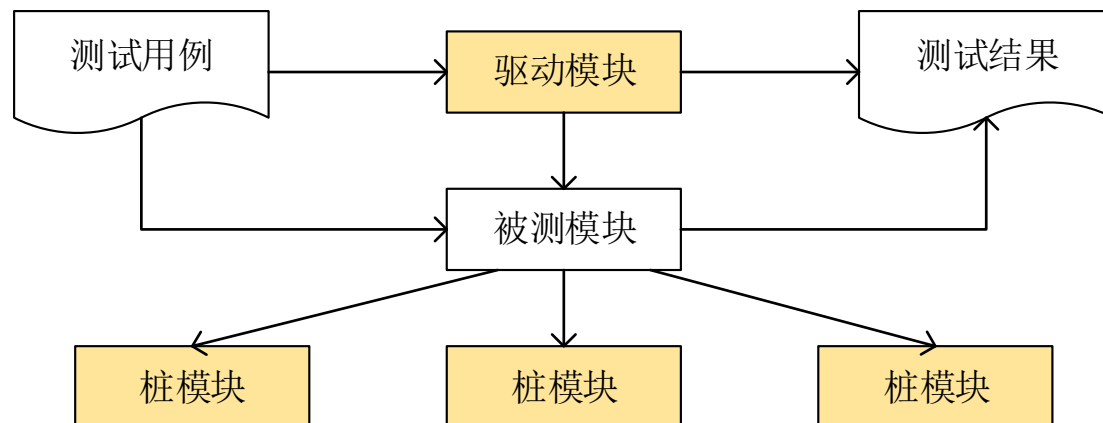
(1) 人工检查

人工检查源程序可以由编写者本人非正式的进行，也可以由审查小组正式进行，后者称为代码审查，是一种非常有效的程序验证技术，对于典型的程序来说，可以查处30-70%的逻辑设计错误和编码错误。

审查小组的组成：组长；程序的设计者；编写者；测试者。

计算机测试

- 在源程序代码编制完成，确认没有语法错误之后，就开始进行单元测试的测试用例设计。程序单元并不一定是一个独立的程序，在考虑测试单元时，同时要考虑它和外界的联系，用一些辅助单元去模拟与被测单元相联系的其它单元。
- 这些辅助单元分为两种：驱动模块和桩模块。桩模块是指模拟被测试的模块所调用的模块，而不是软件产品的组成的部分。驱动模块是模拟调用被测模块的模块。。



集成测试

集成测试也叫组装测试，是将多个模块集成起来成为一个功能进行整体测试，集成测试过程中要考虑如下问题：

- 数据穿过模块接口时是否会丢失；
- 模块的功能是否会对其它模块的功能产生不利的影响；
- 把子功能组合起来，能否达到预期的主功能要求；
- 单个模块的误差累积起来是否会放大到不能接受的程度；
- 全局数据结构是否有问题。

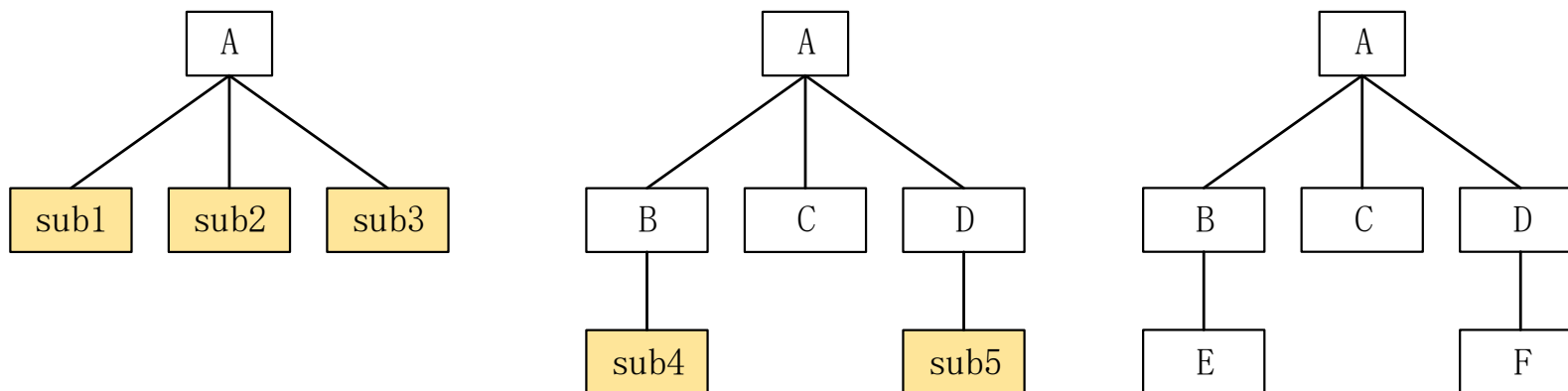
集成测试

集成是组装软件的系统技术；模块组装成程序时有两种方法：

- 1、先分别测试每个模块，再把所有模块按设计要求放在一起结合成所要的程序，称为**非渐增式测试方法**；
 - 2、把下一个要测试的模块同已经测试好的那些模块结合起来进行测试，测试完以后再把下一个该测试的模块结合进来测试，这种每次增加一个模块的方法称为**渐增式测试**（实际上同时完成了单元测试和集成测试）。
- 当使用渐增式的测试方法把模块结合到软件系统中去时，有自顶向下和自底向上两种方法。

集成测试

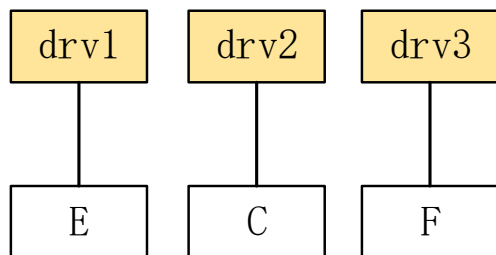
● 自顶向下增式集成测试



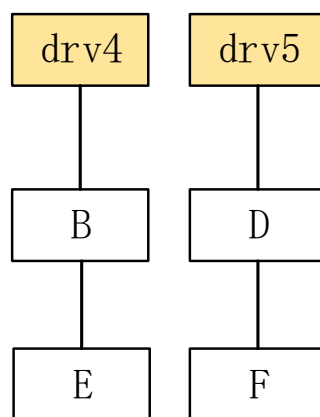
进行单元测试，这时需配以桩模块sub1、sub2和sub3（见图（a）），以模拟被它调用的模块B、C和D。其后，把模块B、C和D与顶层模块A连接起来，再对模块B和D配以桩模块sub4和sub5以模拟对模块E和F的调用。这样按图（b）的形式完成测试。最后，去掉桩模块sub4和sub5，把模块E和F连上即对完整的结构图（见图（c））进行测试。

集成测试

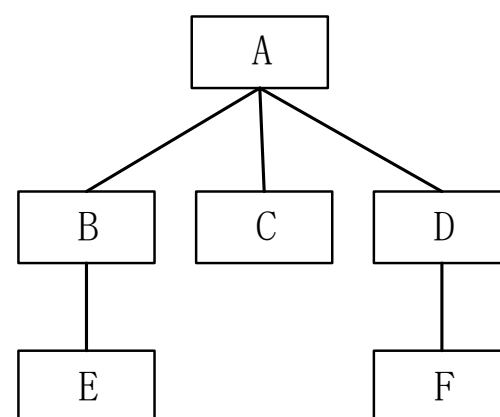
• 自底向上增式集成测试



(a)



(b)



(c)

图 (a)、(b) 和 (c) 表示：树状结构图中处在最下层的叶结点模块E、C和F，由于它们不再调用其他模块，对它们进行单元测试时，只需配以驱动模块drv1、drv2和drv3，用来模拟B、A和D对它们的调用。完成这三个单元测试以后，再按图 (d) 和 (e) 的形式，分别将模块B和E及模块D和F连接起来，在配以驱动模块drv4和drv5的条件下实施部分集成测试。最后再按图 (f) 的形式完成整体的集成测试。

5.4 确认测试

- 确认测试又称有效性测试，目的是向未来的用户表明系统能够象预定要求那样工作，即确认“是否构造了正确的产品？”
- 确认测试一般采用黑盒测试法，测试数据采用用户的真实数据。确认测试有两种可能的结果：
 - (1) 功能和性能与用户需求一致，软件是可以接受的；
 - (2) 功能和性能与用户的要求有差距。

5.5 系统测试

- 系统测试是将通过确认测试的软件，作为整个基于计算机系统的一个元素，与计算机硬件、外设、某些支持软件、数据和人员等其它系统元素结合在一起，在实际运行环境下，对计算机系统进行一系列的组装测试和确认测试。
- 系统测试的目的在于通过与系统的需求定义作比较，发现软件与系统的定义不符合或与之矛盾的地方。

系统测试

(1) 性能测试

性能测试用来测试软件在集成系统中的运行性能，特别是针对实时系统、嵌入式系统。

(2) 强度测试

强度测试是检查当系统运行在高负荷、高并发等情况下，系统可以运行到何种程度的测试。

(3) 安全性测试

安全测试的目的在于验证安装在系统内的保护机制能否在实际中保护系统且不受非法侵入，不受各种非法的干扰。

(4) 安转测试。

确保该软件在正常情况和异常情况下都能进行安装。

系统测试

(5) 恢复测试

通过各种手段，强制性地使软件出错，检查软件的恢复能力。

(6) 互连测试

互连测试是要验证两个或多个不同的系统之间的互连性。

(7) 兼容性测试

兼容性测试验证软件产品在不同版本之间的兼容性。有两类基本的兼容性测试：向下兼容和交错兼容。

性能测试

- 性能测试是要检查系统是否满足在需求说明书中规定的性能。特别是对于实时系统或嵌入式系统。
- 性能测试常常需要与强度测试结合起来进行，并常常要求同时进行硬件和软件检测。
- 通常，对软件性能的检测表现在以下几个方面：响应时间、吞吐量、辅助存储区，例如缓冲区，工作区的大小等、处理精度，等等。

强度测试

强度测试是要检查在系统运行在高负荷、高并发等情况下，系统可以运行到何种程度的测试。

例如：

- 把输入数据速率提高一个数量级，确定输入功能将如何响应。
- 设计需要占用最大存储量或其它资源的测试用例进行测试。
- 设计出在虚拟存储管理机制中引起“颠簸”的测试用例进行测试。
- 设计出会对磁盘常驻内存的数据过度访问的测试用例进行测试。

恢复测试

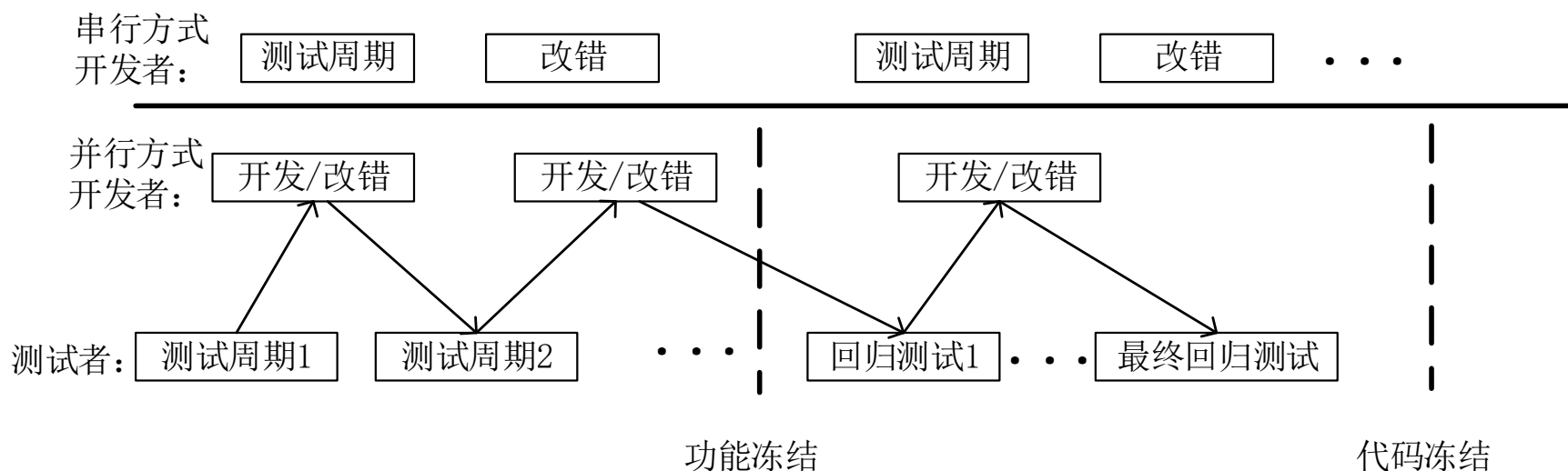
恢复测试是要证实在**克服硬件故障**（包括掉电、硬件或网络出错等）后，**系统能否正常地继续进行工作**，并不对系统造成任何损害。

- 为此，可采用各种人工干预的手段，模拟硬件故障，故意造成软件出错。并由此检查：
 - **错误探测功能**——系统能否发现硬件失效与故障；
 - 能否切换或启动备用的硬件；
 - 在故障发生时能否保护正在运行的作业和系统状态；
 - 在系统恢复后能否从最后记录下来的无错误状态开始继续执行作业，等等。
 - 掉电测试：

6. 软件测试周期

6.1 测试周期

- 软件测试的周期性是指“测试->改错->再测试->再改错”这样一个循环过程，并可分为串行方式和并行方式。
- 采用开发者和测试者并行工作的方式，可以提高工作的效率。



Bug地狱

- 现实中常常是开发人员任务很重，因此开发人员总是想着必须先把新功能全部实现后再修复Bug。如果开发人员认为开发新功能的优先级远大于修复Bug，这样会导致未修复的Bug越来越多，而测试人员无事可做。针对这种情况，可以采用称为Bug地狱 (Bug Hell)的方法。
- 如果某位开发人员积累的未修复的Bug数量超过一个规定值（阈值），则这个开发人员被进入“Bug地狱”，在地狱中，他唯一能做的就是修复Bug，直到Bug数量低于阈值。
- 这一阈值由团队根据实际情况来确定。要注意开发人员同时“入狱”的人数应在全体成员的5%~30%之间

6.2 测试停止

- 第一类标准：测试超过了预定时间，则停止测试。
- 第二类标准：基于测试用例的原则。在功能测试用例通过率达到100%，非功能性测试用例达到95%以上，允许正常结束测试。
- 第三类标准：随着测试的执行，软件已经满足了验收测试指定的功能和非功能性需求，则可以停止测试。
- 第四类标准：根据单位时间内查出故障的数量决定是否停止测试。直到发现的故障几乎为零或者单位时间内查出故障的数量小于设定的阈值，此时可以停止测试。
- 第五类标准：正面指出停止测试的具体要求，即停止测试的标准可定义为查出某一预订数目的故障。

7. 安全测试

7.1 软件安全测试

- 软件安全测试是在充分考虑软件安全性问题的前提下进行的测试。
- 因此，安全测试实际上就是一轮多角度、全方位的攻击和反击，其目的就是要抢在真正攻击者之前尽可能多地找到软件中的漏洞，以减少软件未来遭受攻击的可能性。

软件安全测试步骤

- 1、基于前面设计阶段制定的威胁模型，制定测试计划。
- 2、将安全测试的最小组件单位进行划分，并确定组件的输入格式。
- 3、根据各个接口可能遇到的威胁，或者系统的潜在漏洞，对接口进行分级。
- 4、设计测试用例，确定输入数据。
- 5、攻击应用程序，查看攻击效果。
- 6、总结测试结果，提出解决方案。

7.2 模糊测试

- 模糊测试(Fuzz Testing) 常用于检测软件或计算机系统的安全漏洞，是一种软件安全测试技术。
- 它通过监视非预期输入可能产生的异常结果来发现软件问题。其核心思想是自动或半自动的生成随机数据输入到一个程序中，并监视程序异常（如崩溃），以发现可能的程序错误，比如内存泄漏等。
- 模糊测试的技巧在于它是不符合逻辑的，是将尽可能多的把杂乱的数据输入软件中。

7.3 渗透测试

- 渗透测试 (Penetration Test) 指从一个攻击者的角度来检查和审核一个网络或软件安全性的过程。通常由安全工程师尽可能完整地模拟黑客使用的漏洞发现技术和攻击手段，对目标网络/系统/主机/软件的安全性做深入的探测，以期发现和挖掘系统最脆弱的环节和存在的漏洞，然后输出渗透测试报告，并提交给软件或网络所有者。